on this finding, we propose a one-step approximation of the HRM gradient–using the gradient of the last state of each module and treating other states as constant. The gradient path is, therefore,

$$\text{Output head} \rightarrow \text{final state of the H-module} \rightarrow \text{final state of the L-module} \rightarrow \text{input embedding}$$

The above method needs $O(1)$ memory, does not require unrolling through time, and can be easily implemented with an autograd framework such as PyTorch, as shown in Figure 4. Given that each module only needs to back-propagate errors through its most recent local synaptic activity, this approach aligns well with the perspective that cortical credit assignment relies on short-range, temporally local mechanisms rather than on a global replay of activity patterns.

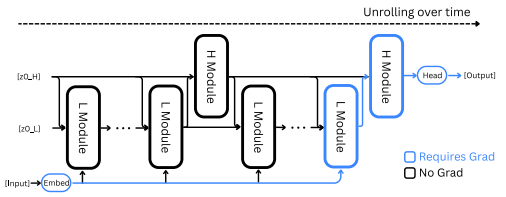
The one-step gradient approximation is theoretically grounded in the mathematics of Deep Equilibrium Models (DEQ)[36] which employs the Implicit Function Theorem (IFT) to bypass BPTT, as detailed next. Consider an idealized HRM behavior where, during high-level cycle k, the L-module repeatedly updates until its state z_L converges to a local fixed point $z_L^\star$. This fixed point, given the current high-level state z_H^{k-1}, can be expressed as

$$z_L^\star = f_L(z_L^\star, z_H^{k-1}, \tilde{x}; \theta_L) \ .$$

The H-module then performs a single update using this converged L-state:

$$z_H^k = f_H(z_H^{k-1}, z_L^\star; \theta_H) \ .$$

With a proper mapping $\mathcal{F}$, the updates to the high-level state can be written in a more compact form as $z_H^k = \mathcal{F}(z_H^{k-1}; \tilde{x}, \theta)$, where $\theta = (\theta_I, \theta_L)$, and the fixed-point can be written as $z_H^\star = \mathcal{F}(z_H^\star; \tilde{x}, \theta)$. Let $J_\mathcal{F} = \frac{\partial \mathcal{F}}{\partial z_H}$ be the Jacobian of $\mathcal{F}$, and assume that the matrix $I - J_\mathcal{F}$ is invertible at $z_H^\star$ and that the mapping $\mathcal{F}$ is continuously differentiable. The Implicit Function Theorem then allows us to calculate the exact gradient of fixed point $z_H^\star$ with respect to the parameters θ without explicit back-propagation:




```
def hrm(z, x, N=2, T=2):
    x = input_embedding(x)
    zH, zL = z

    with torch.no_grad():
        for _i in range(N * T - 1):
            zL = L_net(zL, zH, x)
            if (_i + 1) % T == 0:
                zH = H_net(zH, zL)

    # 1-step grad
    zL = L_net(zL, zH, x)
    zH = H_net(zH, zL)
    return (zH, zL), output_head(zH)

# Deep Supervision
for x, y_true in train_dataloader:
    z = z_init
    for step in range(N_supervision):
        z, y_hat = hrm(z, x)

        loss = softmax_cross_entropy(y_hat, y_true)
        z = z.detach()

        loss.backward()
        opt.step()
        opt.zero_grad()
```

Figure 4: **Top:** Diagram of HRM with approximate gradient. **Bottom:** Pseudocode of HRM with deep supervision training in PyTorch.

$$\frac{\partial z_H^\star}{\partial \theta} = \left(I - J_\mathcal{F}\big|_{z_H^\star}\right)^{-1} \frac{\partial \mathcal{F}}{\partial \theta}\bigg|_{z_H^\star} \ . \tag{1}$$

Calculating the above gradient requires evaluating and inverting matrix $(I - J_\mathcal{F})$ that can be computationally expensive. Given the Neumann series expansion,

$$(I - J_\mathcal{F})^{-1} = I + J_\mathcal{F} + J_\mathcal{F}^2 + J_\mathcal{F}^3 + \dots \ ,$$

the so-called *1-step gradient*[37] approximates the series by considering only its first term, i.e. $(I - J_\mathcal{F})^{-1} \approx I$, and leads to the following approximation of Equation (1):

$$\frac{\partial z_H^*}{\partial \theta_H} \approx \frac{\partial f_H}{\partial \theta_H}, \quad \frac{\partial z_H^*}{\partial \theta_L} \approx \frac{\partial f_H}{\partial z_L^*} \cdot \frac{\partial z_L^*}{\partial \theta_L}, \quad \frac{\partial z_H^*}{\partial \theta_I} \approx \frac{\partial f_H}{\partial z_L^*} \cdot \frac{\partial z_L^*}{\partial \theta_I} \ . \tag{2}$$